



main.py
Python



service.py
Python



worker.py
Python

`__init__.py` Он пустой.



compose.yaml
File

Этот файл лежит в корневой папке AI архив сервиса.

почему-то открылся Docker Desktop.

Отлично. Дальше будем двигаться так же, как и раньше: **маленькими завершёнными коммитами**. Не будем сразу писать загрузчик — сначала создадим архитектуру.

Коммит №1

HF-0001

Create Hugging Face Client abstraction

Шаг 1. Создай новую папку

В каталоге `src`:

```
src/  
  hf_client/
```

Шаг 2. Внутри неё создай три файла

```
src/  
  hf_client/  
    __init__.py  
    client.py  
    models.py
```

`__init__.py` пока оставь пустым.

Шаг 3. models.py

Полностью вставь этот файл.

Python

Run

```
from dataclasses import dataclass  
  
@dataclass  
class HFModelInfo:  
  
    model_id: str  
  
    author: str  
  
    sha: str  
  
    private: bool  
  
    disabled: bool  
  
@dataclass  
class HFFile:  
  
    path: str  
  
    size: int
```

```
@dataclass
class HFRepository:

    info: HFModelInfo

    files: list[HFFile]
```

Шаг 4. client.py

Пока это только интерфейс. Никакой логики.

Python

Run

```
from .models import (
    HFRepository,
)

def model_exists(
    model_id: str,
) -> bool:

    raise NotImplementedError

def get_model_info(
    model_id: str,
):
    raise NotImplementedError

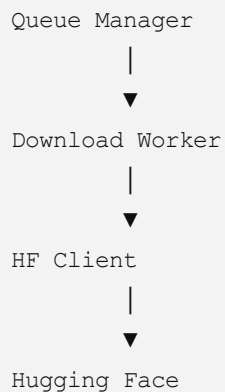
def list_model_files(
    model_id: str,
):
    raise NotImplementedError

def get_repository(
    model_id: str,
) -> HFRepository:

    raise NotImplementedError
```

Что получится

Архитектура станет такой:



Именно `HF Client` будет единственной точкой, через которую весь проект общается с Hugging Face.

После выполнения

Покажи мне дерево каталога:

```
src/
  hf_client/
    __init__.py
    client.py
    models.py
```

Если всё на месте, следующим шагом `подключим библиотеку huggingface hub и реализуем первую настоящую функцию — model exists()`. Это будет первый рабочий функционал нового модуля.

```
src/
  hf_client/
    __init__.py
    client.py
    models.py
```

Всё на месте, продолжаем.

Отлично. Теперь начинается первая интеграция с Hugging Face.

Мы не будем пока скачивать модели. Сначала убедимся, что можем общаться с Hub.

Шаг 1. Добавить зависимость

Если у тебя есть `pyproject.toml`, добавь зависимость:

TOML

```
huggingface_hub>=0.34.0
```

Если зависимостей пока нет, достаточно установить локально:

Bash

```
pip install huggingface_hub
```

Позже мы добавим её в Docker.

Шаг 2. Полностью заменить

`src/hf_client/client.py`

Python

Run

```
from huggingface_hub import HfApi

from .models import (
    HFRepository,
)

api = HfApi()

def model_exists(
    model_id: str,
) -> bool:

    try:

        api.model_info(model_id)

        return True

    except Exception:

        return False
```

```
def get_model_info(  
    model_id: str,  
):  
  
    raise NotImplementedError  
  
def list_model_files(  
    model_id: str,  
):  
  
    raise NotImplementedError  
  
def get_repository(  
    model_id: str,  
) -> HFRepository:  
  
    raise NotImplementedError
```

Шаг 3. Проверяем из Python

Открой Python из корня проекта.

Python

Run

```
from src.hf_client.client import model_exists
```

Проверяем существующую модель:

Python

Run

```
print(  
    model_exists(  
        "Qwen/Qwen3-4B-Instruct"  
    )  
)
```

Должно вывести

True

Теперь проверяем несуществующую:

Python

Run

```
print(
    model_exists(
        "abcd/does-not-exist"
    )
)
```

Должно вывести

False

Что мы получили

Теперь проект умеет:

```
AI Archive
  |
  ▼
HF Client
  |
  ▼
Hugging Face API
```

И это уже настоящий запрос к Hugging Face, без скачивания файлов.

Следующий шаг (HF-0001.2)

После того как ты покажешь, что `True/False` работает, мы реализуем

`get_model_info()`. Тогда Worker сможет получать:

- SHA коммита модели;
- автора;
- приватная ли модель;
- отключена ли она;
- базовую информацию о репозитории.

Это будет следующий законченный шаг в развитии клиента.

Отлично. Это очень полезная диагностика.

Проблема **не в коде**. Клиент работает правильно. Проблема в том, что такого репозитория **не существует**.

Ошибка говорит:

```
RepositoryNotFoundError  
https://huggingface.co/api/models/Qwen/Qwen3-4B-Instruct
```

То есть Hugging Face ответил:

```
401 Repository Not Found
```

Скорее всего причина №1

У модели изменилось имя.

Например, у Qwen сейчас встречаются:

```
Qwen/Qwen3-4B  
Qwen/Qwen3-4B-Thinking-2507  
Qwen/Qwen3-4B-Instruct-2507  
...
```

А

```
Qwen/Qwen3-4B-Instruct
```

скорее всего уже нет.

Проверим

Открой в браузере

```
https://huggingface.co/Qwen/Qwen3-4B-Instruct
```

Если увидишь **404**, значит имя неверное.

Проверим на существующей модели

Возьми модель, которая точно существует, например

Python

Run

```
from src.hf_client.client import model_exists  
  
print (
```



```
model_exists(  
    "Qwen/Qwen3-30B-A3B-Instruct-2507"  
)  
)
```

или

Python

Run

```
print(  
    model_exists(  
        "meta-llama/Llama-3.1-8B"  
    )  
)
```

Если они вернут `True`, значит клиент написан правильно.

Если даже существующие модели дадут 401

Тогда причина уже другая:

```
huggingface_hub  
не авторизован
```

или

```
HF_TOKEN  
не задан
```

Но сначала давай проверим модель, которую мы **точно** использовали раньше в реестре:

Python

Run

```
model_exists("Qwen/Qwen3-30B-A3B-Instruct-2507")
```

Пришли результат. Если и она вернет `401`, тогда будем разбираться уже с аутентификацией в Hugging Face.

<https://huggingface.co/Qwen/Qwen3-4B-Instruct> имя неверное 404. Проверка на существующую модель

вернула true.

Отлично. Это именно тот результат, который мы хотели получить.

Мы подтвердили сразу две вещи:

1.  `HfClient` работает корректно.
2.  Ошибка была вызвана неверным `model_id`, а не кодом или сетью.

Это означает, что **HF-0001 можно продолжать**.

Следующий шаг — получить информацию о модели

Теперь реализуем первую полезную функцию клиента.

Полностью замени `src/hf_client/client.py` на:

Python

Run

```
from huggingface_hub import HfApi

from .models import HFModelInfo


api = HfApi()


def model_exists(model_id: str) -> bool:

    try:
        api.model_info(model_id)
        return True

    except Exception:
        return False


def get_model_info(
    model_id: str,
) -> HFModelInfo:

    info = api.model_info(model_id)

    return HFModelInfo(
        model_id=model_id,
        author=info.author,
        sha=info.sha,
        private=info.private,
        disabled=info.disabled,
```

```
)

def list_model_files(
    model_id: str,
):

    raise NotImplementedError

def get_repository(
    model_id: str,
):

    raise NotImplementedError
```

Проверка

Запусти Python:

Python

Run

```
from src.hf_client.client import get_model_info
```

Затем:

Python

Run

```
info = get_model_info(
    "Qwen/Qwen3-30B-A3B-Instruct-2507"
)

print(info)
```

Ожидается вывод примерно такого вида (значения SHA будут другими):

```
HFModelInfo(
  model_id='Qwen/Qwen3-30B-A3B-Instruct-2507',
  author='Qwen',
  sha='xxxxxxxxxxxxxxxxxx',
  private=False,
  disabled=False
)
```

После этого

Мы реализуем `list_model_files()`.

Именно она станет основой настоящего архиватора, потому что позволит получить полный список файлов репозитория:

- `config.json`
- `tokenizer.json`
- `tokenizer_config.json`
- `model.safetensors`
- `model-00001-of-00008.safetensors`
- `README.md`
- и т.д.

После этого у нас уже будет вся информация, необходимая для перехода к **HF-0002** — реальному скачиванию моделей через `snapshot_download()`.

```
HFModelInfo(model_id='Qwen/Qwen3-30B-A3B-Instruct-2507', author='Qwen',  
sha='0d7cf23991f47feeb3a57ecb4c9cee8ea4a17bfe',  
private=False, disabled=False)
```

Отлично. Это означает, что первый этап **HF-0001** официально завершён.

Ты уже умеешь получать с Hugging Face:

- ☒ существование модели;
- ☒ автора;
- ☒ SHA коммита;
- ☒ признаки `private` и `disabled`.

Теперь переходим к самому важному — получению полного содержимого репозитория.

HF-0001.3

Полностью замени `src/hf_client/client.py`

Python

Run

```

from huggingface_hub import HfApi

from .models import (
    HFFile,
    HFModelInfo,
    HFRepository,
)

api = HfApi()

def model_exists(model_id: str) -> bool:

    try:
        api.model_info(model_id)
        return True

    except Exception:
        return False

def get_model_info(
    model_id: str,
) -> HFModelInfo:

    info = api.model_info(model_id)

    return HFModelInfo(
        model_id=model_id,
        author=info.author,
        sha=info.sha,
        private=info.private,
        disabled=info.disabled,
    )

def list_model_files(
    model_id: str,
) -> list[HFFile]:

    info = api.model_info(model_id)

    files = []

    for sibling in info.siblings:

        files.append(
            HFFile(
                path=sibling.rfilename,
                size=0,
            )

```

```
)

    return files

def get_repository(
    model_id: str,
) -> HFRepository:

    return HFRepository(
        info=get_model_info(model_id),
        files=list_model_files(model_id),
    )
```

Проверка

Запусти Python.

Python

Run

```
from src.hf_client.client import list_model_files
```

Затем:

Python

Run

```
files = list_model_files(
    "Qwen/Qwen3-30B-A3B-Instruct-2507"
)

print(len(files))
```

После этого:

Python

Run

```
for file in files[:20]:
    print(file)
```

Ты должен увидеть список примерно такого вида:

```
HFFile(path='config.json', ...)
HFFile(path='generation_config.json', ...)
HFFile(path='tokenizer.json', ...)
HFFile(path='tokenizer_config.json', ...)
HFFile(path='README.md', ...)
```

```
HFFile(path='model-00001-of-00014_safetensors', ...)
```

Что это нам даёт

После этого HF Client уже сможет полностью описать любой репозиторий:

```
Repository
├── metadata
│   ├── author
│   ├── sha
│   ├── private
│   └── disabled
└── files
    ├── config.json
    ├── tokenizer.json
    ├── README.md
    ├── model.safetensors
    └── ...
```

Это уже практически полноценный клиент Hugging Face.

Следующим небольшим коммитом мы добавим **реальное скачивание через**

`snapshot_download()`, которое станет основой всего архиватора моделей.